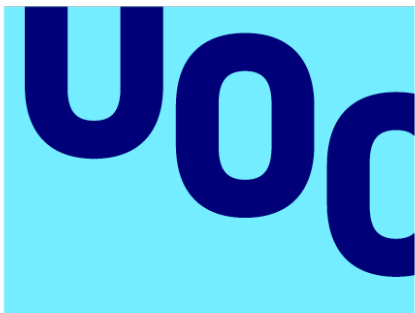


Sistema software de optimización tridimensional de packaging para entornos logísticos



Universitat
Oberta
de Catalunya

Fernando Cabeza Pereira

Nombre del Programa
Desarrollo Web

Nombre Tutor/a de TF

Nombre y apellidos

**Profesor/a responsable de la
asignatura**

Nombre y apellidos



Esta obra está sujeta a una licencia de Reconocimiento-NoComercial-SinObraDerivada [3.0 España de Creative Commons](https://creativecommons.org/licenses/by-nc-nd/3.0/es/)

FICHA DEL TRABAJO FINAL

Título del trabajo:	Sistema software de optimización tridimensional de packaging para entornos logísticos
Nombre del autor:	Fernando Cabeza Pereira
Nombre del director/a:	
Nombre del PRA:	
Fecha de entrega (mm/aaaa):	06/2026
Titulación o programa:	Grado en Ingeniería Informática
Área del Trabajo Final:	Desarrollo web
Idioma del trabajo:	Castellano
Palabras clave	Optimización 3D, Arquitectura de microservicios, Logística
Resumen del Trabajo	
El presente Trabajo Final de Grado tiene como finalidad el diseño e implementación de un sistema software de optimización tridimensional orientado a entornos logísticos. El problema abordado consiste en determinar la disposición más eficiente de objetos dentro de un contenedor, maximizando la ocupación volumétrica y respetando restricciones de peso, orientación y reglas operativas. El sistema se concibe como una solución genérica aplicable a cualquier organización que requiera consolidar elementos físicos dentro de unidades de transporte. La metodología adoptada corresponde a un enfoque profesionalizador, basado en el desarrollo incremental de una arquitectura modular y desacoplada. La solución se implementa mediante una API REST desarrollada en Python (FastAPI), persistencia en base de datos NoSQL (MongoDB), un microservicio independiente para la gestión de usuarios y roles, y un panel administrativo para la parametrización dinámica de reglas de negocio. Asimismo, el sistema incorpora visualización tridimensional interactiva para la validación de resultados. Se espera obtener como resultado una plataforma funcional, escalable y configurable que reduzca el espacio vacío en los envíos y mejore la eficiencia operativa. El proyecto demuestra la viabilidad técnica de aplicar algoritmos de optimización tridimensional integrados en arquitecturas modernas de microservicios.	

Abstract

This Final Degree Project aims to design and implement a three-dimensional packaging optimization software system for logistics environments. The problem addressed involves determining the most efficient arrangement of objects inside a container, maximizing volumetric utilization while respecting weight constraints, orientation rules, and operational requirements.

The system is conceived as a generic solution applicable to any organization that needs to consolidate physical items into transport units. The adopted methodology follows a professional-oriented approach based on incremental development within a modular and decoupled architecture. The solution is implemented using a REST API developed in Python (FastAPI), data persistence through a NoSQL database (MongoDB), an independent microservice for user and role management, and an administrative panel for dynamic configuration of business rules. Additionally, the system integrates an interactive three-dimensional visualization module to validate optimization results.

The expected outcome is a functional, scalable, and configurable platform capable of reducing unused transport volume and improving operational efficiency. The project demonstrates the technical feasibility of integrating three-dimensional optimization algorithms within modern microservices-based architectures.

Índice

1.	Introducción.....	1
1.1.	Contexto y justificación del Trabajo	1
1.2.	Objetivos del Trabajo	2
1.3.	Impacto en sostenibilidad, ético-social y de diversidad	2
1.4.	Enfoque y método seguido	4
1.5.	Planificación del Trabajo	5
1.6.	Breve resumen de productos obtenidos	9
1.7.	Breve descripción de los otros capítulos de la memoria	9
1.8.	Declaración sobre el uso de inteligencia artificial generativa	9
2.	Materiales y métodos	10
2.1.	Enfoque metodológico y estrategia de desarrollo	10
2.2.	Estudio y selección de tecnologías	11
2.3.	Requerimientos del sistema	14
2.3.1.	Actores del sistema.....	14
2.3.2.	Identificación de casos de uso	15
2.4.	Análisis funcional	17
2.4.1.	Caso de uso: Optimizar embalaje	18
2.4.2.	Caso de uso: Revisar propuesta.....	21
2.4.3.	Caso de uso: Gestionar contenedores.....	23
2.5.	Diseño del sistema	24
2.5.1.	Modelo de datos.....	25
2.5.2.	Clases principales del dominio	28
2.5.3.	Arquitectura del sistema.....	30

Lista de figuras

1. Introducción.....	1
Figura 0: Diagrama de flujo de la aplicación.....	4
Figura 1: Diagrama de gant.....	8
2. Materiales y métodos	10
Figura 2: Diagrama de casos de usuario.....	16
Figura 3: Paso 1. Grid de producto	18
Figura 4: Paso 2. Introducción de medidas.	18
Figura 5: Paso 3. Resultado de optimización.	19
Figura 6: Paso 1: Revisión de propuesta.....	21
Figura 8: Diagrama de clases de dominio	27
Figura 8: Diagrama de clases de dominio	29
Figura 9: Diagrama de arquitectura.....	31

1. Introducción

1.1. Contexto y justificación del Trabajo

En la logística de distribución y preparación de envíos, muchas operaciones consisten en agrupar objetos individuales —paquetes, productos o unidades— dentro de un contenedor de transporte para su envío, almacenamiento o consolidación. Aunque el problema puede parecer trivial a pequeña escala, cuando el volumen de artículos, la variedad de geometrías (dimensiones), las restricciones de peso, y las condiciones operativas aumentan, la elección del contenedor y la disposición interna de los objetos se convierte en un problema complejo con impacto directo en costes, sostenibilidad y eficiencia operativa.

Actualmente, la toma de decisiones sobre qué contenedor utilizar y cómo organizar los objetos suele apoyarse en reglas empíricas (reglas de pulgar), plantillas manuales o procesos semiautomáticos que no explotan de forma rigurosa la información geométrica y de peso disponible. Esto puede provocar: ocupación ineficiente del volumen útil, uso excesivo de material de transporte, peores costes de envío por ocupación subóptima, mayor consumo energético por transporte de “aire” y mayor riesgo de incumplimiento de restricciones de peso/fragilidad.

El proyecto propone desarrollar un **software de optimización de packaging** genérico capaz de, dado un conjunto de objetos con sus dimensiones, peso y restricciones (orientación, apilado, fragilidad, prioridad, lotes, etc.), calcular automáticamente:

- el contenedor exterior más eficiente y compatible con las restricciones; y
- una disposición interna optimizada de los objetos que maximice la ocupación útil y minimice el espacio vacío y el riesgo por exceso de peso o incumplimiento de reglas.

El alcance funcional del sistema se orienta a entornos industriales y logísticos con volúmenes medios/altos (centros de distribución, almacenes de preparación de pedidos, operaciones de consolidación), pero manteniendo suficiente generalidad para aplicarse a otros dominios que muevan “cosas” dentro de contenedores (p. ej., paquetería, retail, e-commerce, inmuebles logísticos de terceros). El sistema se diseñará como un servicio web escalable, con APIs que permitan integrarlo en procesos automatizados y con una interfaz visual para analizar y validar las propuestas de packaging.

Además de la eficiencia operativa, el proyecto atiende a criterios de sostenibilidad (reducción de volumen transportado por unidad, menor consumo logístico) y de trazabilidad (registro y versionado de propuestas), aportando tanto valor económico como cumplimiento y control del proceso.

1.2. Objetivos del Trabajo

1. Diseñar un modelo de datos que permita representar objetos, contenedores y restricciones de forma flexible y extensible.
2. Implementar un algoritmo de optimización tridimensional que calcule automáticamente la disposición más eficiente de los objetos dentro de un contenedor.
3. Desarrollar una API REST utilizando FastAPI que exponga los servicios de cálculo y consulta de resultados.
4. Integrar un sistema de persistencia basado en MongoDB para almacenar configuraciones, propuestas generadas e histórico de operaciones.
5. Diseñar e implementar un microservicio independiente para la gestión de usuarios, roles y control de acceso.
6. Desarrollar un panel administrativo que permita parametrizar reglas de negocio, definir tipologías de contenedores y establecer prioridades operativas.
7. Incorporar un módulo de visualización tridimensional interactiva que facilite la validación y análisis de las soluciones generadas.
8. Evaluar el rendimiento y la eficiencia del sistema mediante métricas objetivas de ocupación volumétrica y reducción de espacio vacío.

1.3. Impacto en sostenibilidad, ético-social y de diversidad

El desarrollo del sistema propuesto presenta impactos relevantes en las dimensiones ambiental, ético-social y de diversidad, alineados con la competencia transversal de compromiso ético y global.

Impacto en sostenibilidad ambiental

El principal impacto positivo del proyecto se relaciona con la mejora de la eficiencia volumétrica en los procesos logísticos. La optimización en la disposición de objetos dentro de contenedores permite reducir el espacio vacío en los envíos, lo que puede traducirse en:

- Menor número de unidades de transporte necesarias.
- Reducción del consumo de combustible asociado al transporte.
- Disminución indirecta de emisiones de CO₂.
- Optimización del uso de materiales de embalaje.

Aunque el proyecto no cuantifica directamente la reducción de emisiones, sí proporciona una herramienta que favorece decisiones logísticas más eficientes y sostenibles.

Impacto ético-social

Desde el punto de vista ético, el sistema promueve la mejora de la eficiencia operativa sin sustituir la toma de decisiones humanas, sino apoyándola con herramientas basadas en datos. Se plantea como un sistema de apoyo a la decisión y no como un mecanismo autónomo que elimine supervisión humana.

En términos de responsabilidad tecnológica:

- Se garantizará la protección de datos de usuarios mediante mecanismos de autenticación y control de acceso.
- El diseño modular permitirá auditoría y trazabilidad de las decisiones generadas por el sistema.
- No se procesarán datos personales sensibles más allá de la información básica necesaria para la gestión de usuarios.

Impacto en diversidad e inclusión

El sistema no incorpora sesgos relacionados con variables personales, ya que su funcionamiento se basa exclusivamente en parámetros físicos y operativos (dimensiones, peso, restricciones técnicas).

Asimismo, la interfaz del sistema podrá diseñarse siguiendo principios básicos de accesibilidad digital, facilitando su uso por perfiles diversos dentro de la organización.

Impactos negativos potenciales y mitigación

Entre los posibles impactos negativos se identifica la dependencia tecnológica excesiva en procesos críticos. Para mitigar este riesgo, el sistema se concibe como herramienta de apoyo, permitiendo revisión y validación humana de los resultados generados.

1.4. Enfoque y método seguido

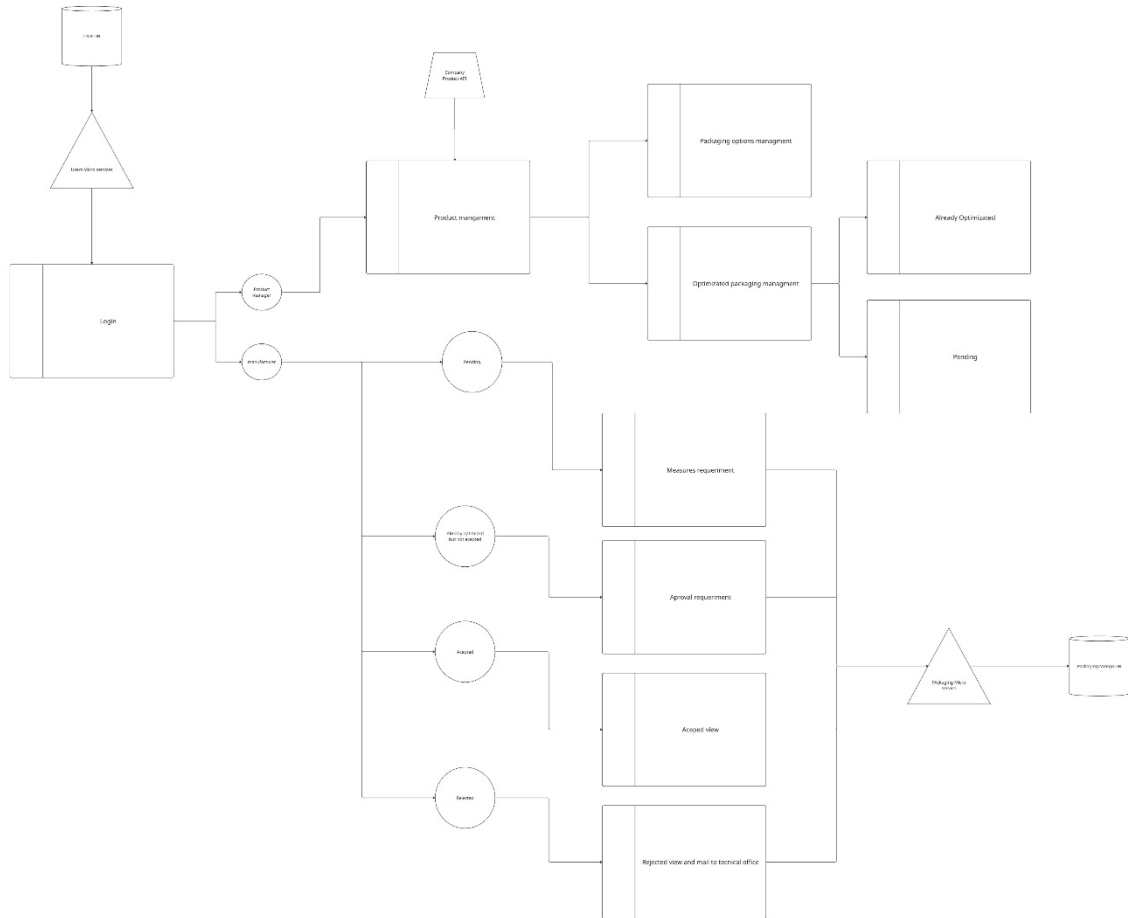


Figura 0: Diagrama de flujo de la aplicación.

El presente Trabajo Final se plantea como un Trabajo Final Profesionalizador (TFP), orientado al diseño e implementación de una solución tecnológica completa que abarca tanto el backend como la capa de presentación.

El enfoque adoptado prioriza la obtención de un producto funcional, modular y escalable, integrando conocimientos adquiridos en ingeniería del software, arquitectura de sistemas distribuidos, desarrollo backend y desarrollo frontend moderno.

Estrategia de desarrollo

Se adopta un enfoque incremental basado en la implementación progresiva de módulos independientes que posteriormente se integran en una arquitectura común. Este enfoque permite validar cada componente de manera autónoma antes de su integración global.

El sistema se estructura en tres grandes bloques:

1. Motor de optimización y servicios backend.
2. Sistema de persistencia y control de acceso.
3. Aplicación frontend para interacción de usuarios y administración.

Arquitectura general

La solución sigue una arquitectura basada en microservicios y separación clara entre frontend y backend:

- Servicio principal de optimización, encargado del cálculo tridimensional.
- Base de datos MongoDB para almacenamiento de configuraciones y resultados.
- Microservicio independiente para la gestión de usuarios y roles.
- API REST desarrollada con FastAPI para exponer los servicios.
- Aplicación frontend desarrollada en React para la interacción con el sistema.

Capa de presentación

La aplicación frontend se implementará mediante React, diferenciando dos perfiles funcionales:

Interfaz operativa

Permite a los usuarios introducir las dimensiones y características de los objetos a consolidar, ejecutar la optimización y visualizar los resultados en formato tridimensional interactivo, así como consultar métricas asociadas.

Panel administrativo

Permite definir y parametrizar:

- Tipologías de contenedores disponibles.
- Rangos y prioridades.
- Restricciones específicas según categoría de producto.
- Casos especiales de configuración.

Esta separación funcional garantiza una gestión estructurada de las reglas de negocio y facilita la adaptabilidad del sistema a distintos entornos.

Metodología técnica

El desarrollo se apoyará en:

- Python y FastAPI para backend.
- MongoDB como sistema de persistencia.
- React para la capa frontend.
- Principios de diseño modular y separación de responsabilidades.
- Control de versiones mediante Git.

1.5. Planificación del Trabajo

El proyecto se desarrollará durante un semestre académico de 16 semanas, estructurado conforme a las pruebas de evaluación continua (PEC) establecidas en la asignatura.

La planificación combina la organización académica del TFG con la estructura técnica del proyecto.

PEC 1 – Propuesta, diseño y planificación (Semanas 1–3)

Objetivo académico:

Definir el problema, contextualización, objetivos, impacto, metodología y planificación.

Trabajo técnico realizado:

- Análisis detallado del problema.
- Definición del alcance funcional.
- Diseño preliminar de arquitectura.
- Modelado conceptual del sistema.
- Identificación de riesgos.
- Planificación temporal (Gantt).

Entregable:

Documento de Plan de Trabajo.

PEC 2 – Desarrollo del proyecto (Parte 1) (Semanas 4–8)

Objetivo académico:

Desarrollo inicial del sistema y validación parcial.

Trabajo técnico realizado:

- Implementación del modelo tridimensional.
- Desarrollo del motor de optimización.
- Diseño de estructura de datos.
- Primeros endpoints REST.
- Pruebas unitarias del núcleo lógico.
- Documentación parcial de avances.

Entregable:

Versión funcional del motor de optimización y backend inicial.

PEC 3 – Desarrollo del proyecto (Parte 2) (Semanas 9–13)

Objetivo académico:

Completar el desarrollo y consolidar el sistema.

Trabajo técnico realizado:

- Integración completa con MongoDB.

- Implementación del microservicio de usuarios y roles.
- Desarrollo de autenticación y autorización.
- Desarrollo frontend en React:
 - Interfaz operativa.
 - Panel administrativo.
- Integración frontend–backend.
- Validación de casos complejos.

Entregable:

Sistema completo funcional e integrado.

PEC 4 – Memoria y presentación (Semanas 14–15)

Objetivo académico:

Redactar la memoria final y preparar la presentación.

Trabajo realizado:

- Redacción completa de capítulos:
 - Materiales y métodos.
 - Resultados.
 - Conclusiones.
- Elaboración de diagramas:
 - Arquitectura.
 - Modelo de datos.
 - Flujo de optimización.
- Generación de índices automáticos.
- Preparación de presentación de 20 minutos.
- Ensayo de defensa.

Entregable:

Memoria final y vídeo de presentación.

PEC 5 – Defensa (Semana 16)

Objetivo académico:

Responder a preguntas del tribunal y defender el trabajo realizado.

Trabajo realizado:

- Revisión final del proyecto.
- Preparación de respuestas técnicas.
- Defensa ante la comisión evaluadora.

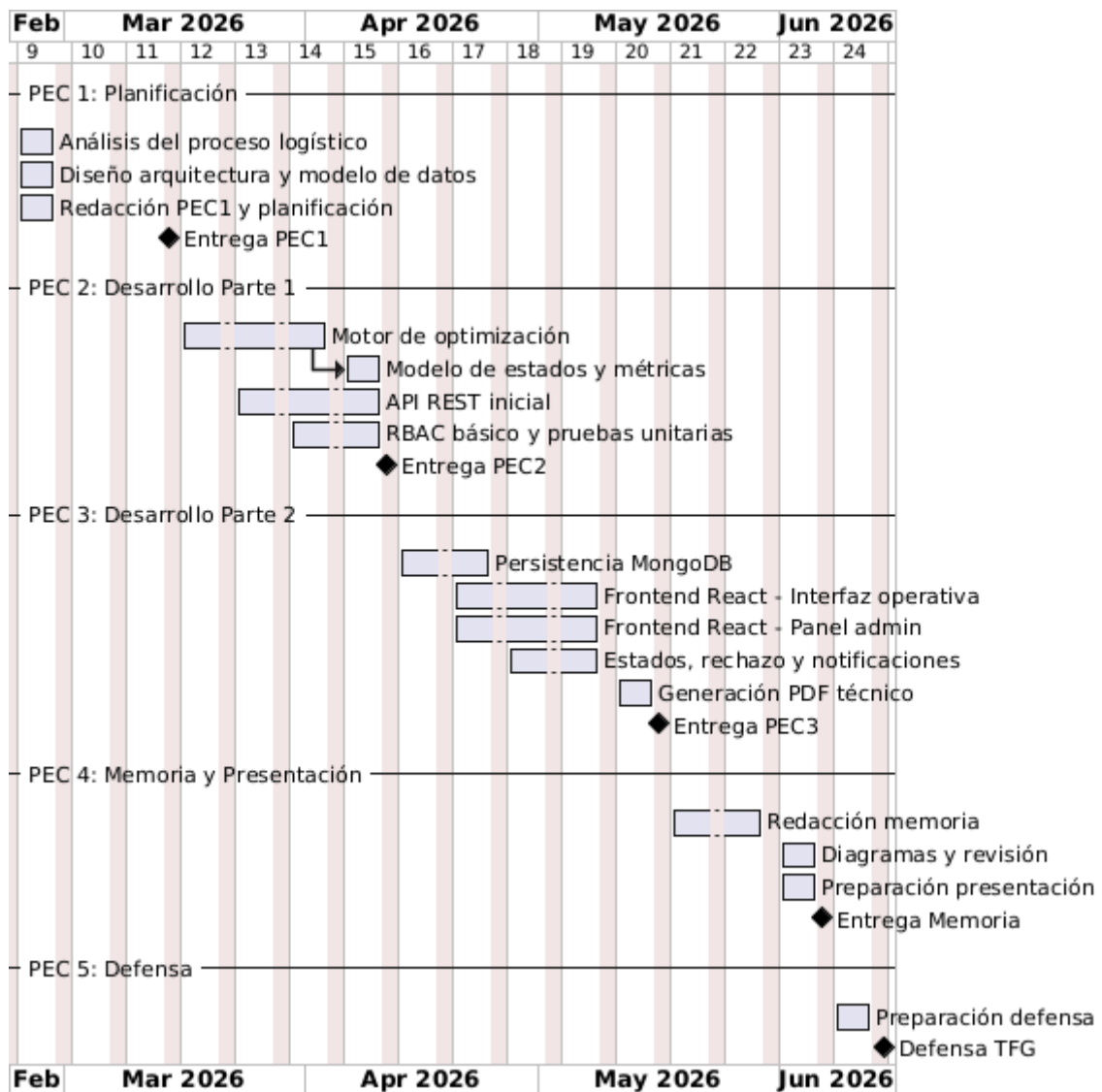


Figura 1: Diagrama de gant

1.6. Breve resumen de productos obtenidos

En el desarrollo del presente trabajo se ha definido el diseño completo de un sistema software de optimización tridimensional de packaging. Como resultado, se ha establecido una arquitectura basada en microservicios, un modelo de datos orientado a documentos, así como la definición de los principales casos de uso y flujos funcionales del sistema.

Asimismo, se ha desarrollado un prototipo funcional que permite validar el proceso de optimización, incluyendo la interacción del usuario, la visualización de resultados y la gestión de configuraciones operativas.

1.7. Breve descripción de los otros capítulos de la memoria

El capítulo 1 introduce el contexto del problema, los objetivos del trabajo y el enfoque metodológico adoptado.

El capítulo 2 describe los materiales y métodos utilizados, incluyendo el análisis de requerimientos, los casos de uso, el diseño del sistema, el modelo de datos, las clases del dominio y la arquitectura propuesta.

Los capítulos posteriores abordarán los resultados obtenidos, el análisis de las conclusiones y las posibles líneas de trabajo futuro derivadas del proyecto.

1.8. Declaración sobre el uso de inteligencia artificial generativa

Durante la elaboración del presente Trabajo Final se ha utilizado una herramienta de inteligencia artificial generativa (ChatGPT) como apoyo en tareas de redacción y estructuración del documento. Su uso se ha limitado a funciones de asistencia lingüística y organizativa, sin delegar en ella la toma de decisiones técnicas ni el diseño conceptual del sistema.

En concreto, la herramienta se ha empleado para:

- Refinar la redacción de apartados.
- Estructurar secciones del documento.
- Revisar la coherencia y claridad del contenido técnico.
- Generar propuestas de esquemas y diagramas para su posterior adaptación por parte del autor.

Todas las decisiones de arquitectura, alcance y desarrollo han sido adoptadas por el autor, quien asume la plena responsabilidad académica e intelectual del trabajo presentado.

2. Materiales y métodos

2.1. Enfoque metodológico y estrategia de desarrollo

El desarrollo del presente trabajo se plantea siguiendo un enfoque incremental orientado a la construcción de un producto software funcional, alineado con la naturaleza profesionalizadora del proyecto.

En lugar de adoptar una metodología estrictamente secuencial, se opta por una estrategia iterativa que permite abordar de forma progresiva los distintos componentes del sistema. Este enfoque facilita la validación continua de las decisiones de diseño, así como la adaptación a las necesidades detectadas durante el desarrollo del proyecto.

El sistema se estructura en diferentes bloques funcionales que se desarrollan de manera progresiva:

- Definición del modelo de datos y de las entidades principales.
- Diseño del motor de optimización tridimensional.
- Definición de la API REST para la gestión de operaciones y estados.
- Diseño de los microservicios auxiliares (usuarios y productos).
- Diseño de la interfaz de usuario y del panel administrativo.
- Integración de los distintos componentes del sistema.

Este enfoque permite disponer de una visión global del sistema desde fases tempranas, facilitando la coherencia entre los distintos módulos y asegurando que las decisiones de diseño se alineen con los objetivos funcionales del proyecto.

Asimismo, se plantea el uso de control de versiones mediante Git para la gestión del desarrollo, permitiendo organizar el trabajo en distintas ramas funcionales y garantizar la trazabilidad de los cambios.

En cuanto a la validación, se contempla la realización de pruebas sobre los componentes críticos del sistema, así como pruebas de integración para verificar el correcto funcionamiento del flujo completo.

2.2. Estudio y selección de tecnologías

Para el desarrollo del sistema se ha realizado un estudio de diferentes tecnologías y herramientas, con el objetivo de seleccionar aquellas que mejor se adaptan a los requisitos funcionales y no funcionales del proyecto, tales como la escalabilidad, la modularidad, la capacidad de integración y la facilidad de mantenimiento.

La solución propuesta se basa en una arquitectura de microservicios, lo que implica la necesidad de seleccionar tecnologías adecuadas tanto para el desarrollo backend como frontend, así como para la persistencia de datos y la orquestación del sistema.

Backend: Python y FastAPI

Para la implementación de los servicios backend se selecciona Python como lenguaje principal debido a su amplia adopción, su ecosistema de librerías y su idoneidad tanto para el desarrollo de aplicaciones web como para la implementación de lógica algorítmica.

En particular, Python resulta especialmente adecuado para este proyecto al permitir integrar de forma natural el desarrollo del motor de optimización tridimensional junto con la capa de servicios, evitando la necesidad de utilizar diferentes lenguajes para el cálculo y la exposición de la funcionalidad.

Como framework web se opta por FastAPI, una solución moderna que permite el desarrollo de APIs REST de alto rendimiento. Entre los principales motivos de esta elección destacan:

- Soporte nativo para programación asíncrona.
- Integración automática con OpenAPI para la documentación de la API (enfoque contract-first).
- Validación de datos mediante modelos tipados (Pydantic).
- Alto rendimiento en comparación con otros frameworks Python tradicionales.

Esta combinación permite definir interfaces claras entre servicios, facilitar la comunicación entre microservicios y asegurar la coherencia entre la lógica de negocio y los servicios expuestos.

Persistencia: MongoDB

Para la capa de datos se selecciona MongoDB como sistema de gestión de bases de datos NoSQL orientado a documentos.

Esta elección se justifica por la naturaleza del modelo de datos del sistema, que incluye estructuras flexibles y anidadas, como:

- Dimensiones de objetos y contenedores.
- Configuración de reglas de negocio.

- Propuestas de optimización con múltiples resultados evaluados.

El uso de un modelo documental permite evitar estructuras rígidas y facilita la evolución del esquema de datos sin necesidad de migraciones complejas, lo cual resulta especialmente adecuado en un sistema con alta variabilidad en sus entidades.

Frontend: React, TypeScript y Vite

Para la capa de presentación se selecciona React como librería principal para el desarrollo de interfaces de usuario, debido a su enfoque basado en componentes y su amplia adopción en entornos profesionales.

Se complementa con TypeScript, que aporta tipado estático, mejorando la mantenibilidad del código y reduciendo errores en tiempo de desarrollo.

Como herramienta de construcción se utiliza Vite, que proporciona:

- Arranque rápido del entorno de desarrollo.
- Recarga en caliente (Hot Module Replacement).
- Configuración sencilla y optimizada para proyectos modernos.

Esta combinación permite desarrollar una aplicación de tipo SPA (Single Page Application) con una experiencia de usuario fluida.

Contenerización y despliegue: Docker Compose

Para la ejecución del sistema se plantea el uso de Docker Compose, que permite orquestrar los distintos servicios del sistema (microservicios backend, base de datos y frontend) en un entorno unificado.

Las principales ventajas de esta elección son:

- Reproducibilidad del entorno de desarrollo.
- Aislamiento de servicios.
- Facilidad para simular entornos de despliegue reales.
- Simplificación de la configuración de dependencias.

Autenticación y control de acceso: JWT y RBAC

El sistema incorpora un mecanismo de autenticación basado en tokens JWT (JSON Web Tokens), que permite gestionar sesiones de forma stateless.

Además, se define un modelo de control de acceso basado en roles (RBAC), que diferencia entre los perfiles principales del sistema:

- Fabricante / proveedor

- Gestor de producto / operaciones
- Administrador

Este enfoque permite controlar el acceso a las distintas funcionalidades del sistema de forma flexible y escalable.

Visualización y generación de informes

Para la representación de resultados se plantea el uso de herramientas de visualización tridimensional, permitiendo mostrar la disposición de los objetos dentro del contenedor de forma interactiva.

Asimismo, se contempla la generación de informes en formato PDF que incluyen:

- Representación gráfica de la solución.
- Métricas de eficiencia.
- Detalles de la configuración seleccionada.

Estas funcionalidades permiten mejorar la comprensión de los resultados y facilitan la validación por parte de los usuarios.

Para la representación tridimensional de las soluciones se utiliza la librería Plotly en el backend, que permite generar visualizaciones interactivas en formato HTML a partir de los resultados del motor de optimización. Estas representaciones son posteriormente enviadas al frontend, que se encarga de su visualización.

Esta aproximación permite desacoplar la lógica de generación de la visualización de su presentación, facilitando la reutilización de los resultados tanto en la interfaz web como en los informes generados, y proporcionando al usuario una herramienta visual clave para la validación de las soluciones propuestas.

2.3. Requerimientos del sistema

El sistema propuesto tiene como objetivo dar soporte al proceso de optimización tridimensional de embalaje en entornos logísticos, permitiendo a los distintos perfiles de usuario interactuar con la plataforma para generar, validar y gestionar propuestas de embalaje.

Los requerimientos funcionales del sistema se definen a partir de las necesidades de los usuarios que interactúan con la aplicación, así como de los flujos operativos que se desean soportar.

2.3.1. Actores del sistema

Se identifican tres tipos principales de actores que interactúan con el sistema:

Fabricante / proveedor

Usuario encargado de introducir la información de los productos (dimensiones, características, lotes) y de solicitar la optimización del embalaje. Además, puede visualizar los resultados generados, descargar informes y aceptar o rechazar las propuestas.

Gestor de producto / operaciones

Usuario responsable de supervisar las propuestas generadas por los fabricantes, revisar su validez, y gestionar la configuración del sistema, incluyendo contenedores, reglas de embalaje y prioridades operativas.

Administrador

Usuario con permisos completos sobre el sistema, encargado de la gestión de usuarios, roles y configuración global. Dispone de acceso a todas las funcionalidades del sistema.

2.3.2. Identificación de casos de uso

A partir de los actores identificados, se definen los principales casos de uso del sistema, entendidos como las interacciones que aportan valor al usuario final.

Fabricante / proveedor

- UC01 Autenticarse en el sistema
- UC02 Gestionar productos propios
- UC03 Optimizar embalaje
- UC04 Visualizar resultado de la optimización (incluyendo representación 3D)
- UC05 Descargar informe en formato PDF
- UC06 Aceptar propuesta de embalaje
- UC07 Rechazar propuesta de embalaje

Gestor de producto / operaciones

- UC01 Autenticarse en el sistema
- UC08 Revisar propuestas de embalaje
- UC09 Gestionar contenedores (alta, baja, modificación, consulta)
- UC10 Gestionar reglas de embalaje
- UC11 Asignar reglas a familias o subfamilias de productos

Administrador

- UC01 Autenticarse en el sistema
- UC12 Gestionar usuarios (alta, baja, modificación, asignación de roles)

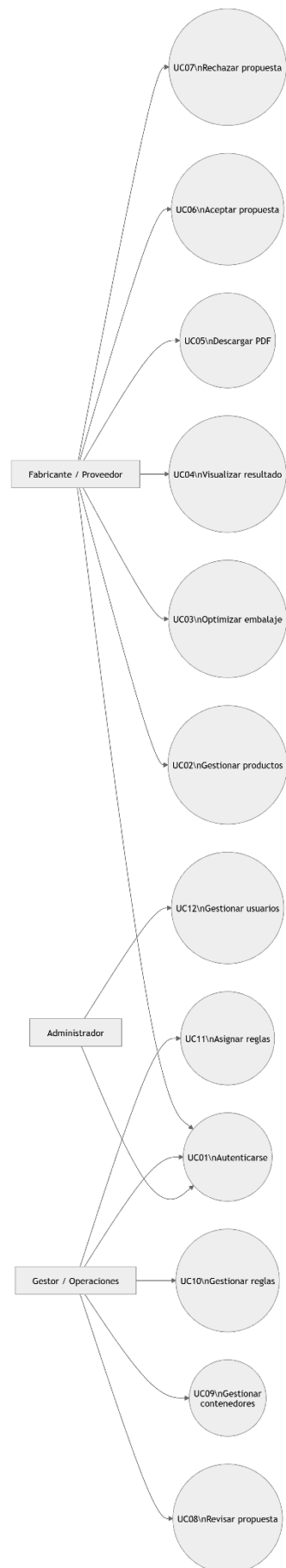


Figura 2: Diagrama de casos de usuario.

2.4. Análisis funcional

En este apartado se describen los principales casos de uso del sistema, centrándose en aquellos que presentan mayor complejidad funcional o relevancia dentro del flujo operativo.

Para cada caso de uso se detallan las condiciones necesarias para su ejecución, así como el flujo principal de interacción entre el usuario y el sistema, incluyendo posibles escenarios alternativos.

2.4.1. Caso de uso: Optimizar embalaje

Actor principal: Fabricante / proveedor

Descripción:

Este caso de uso permite al usuario introducir las características de un producto y solicitar al sistema el cálculo de la configuración de embalaje óptima.

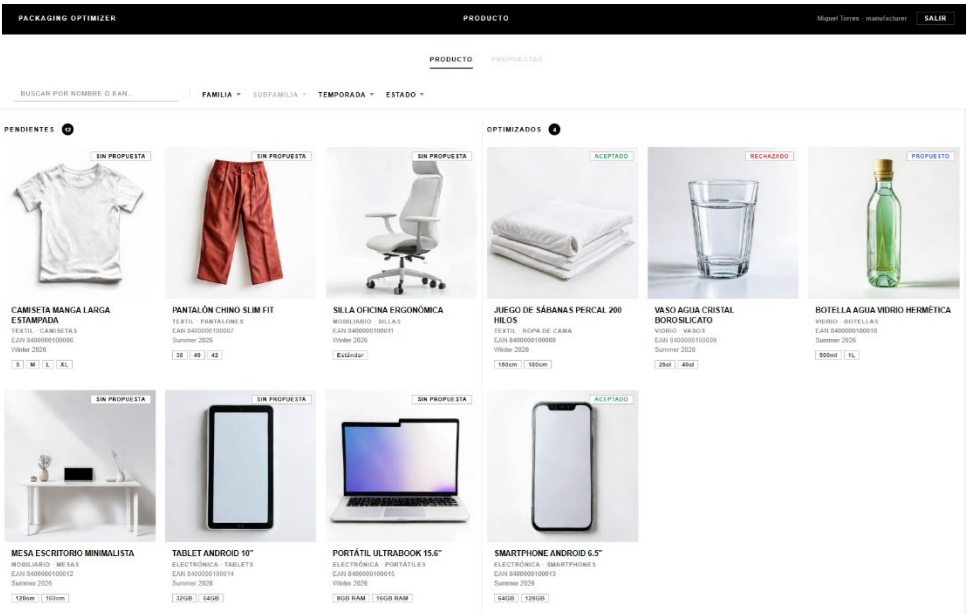


Figura 3: Paso 1. Grid de producto

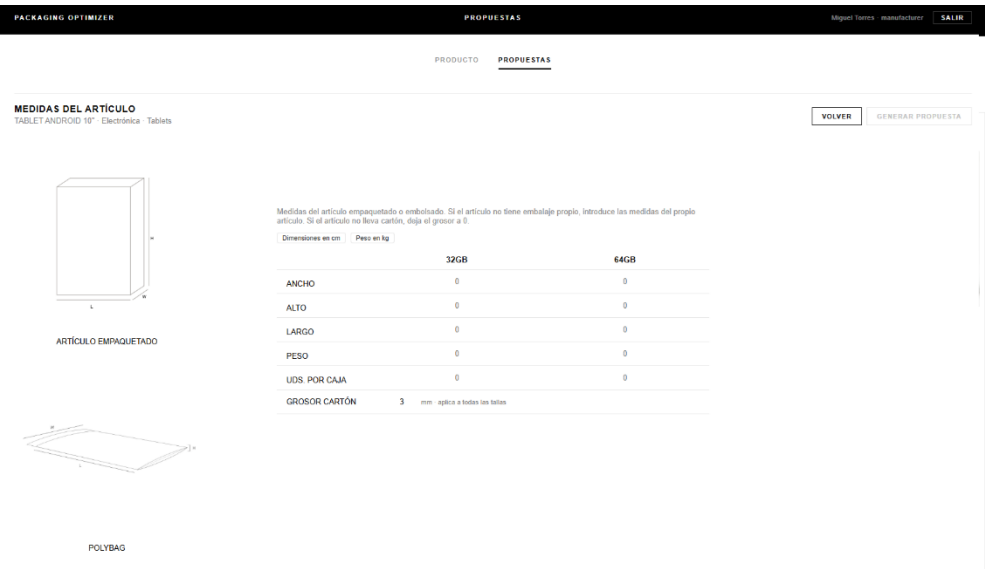


Figura 4: Paso 2. Introducción de medidas.

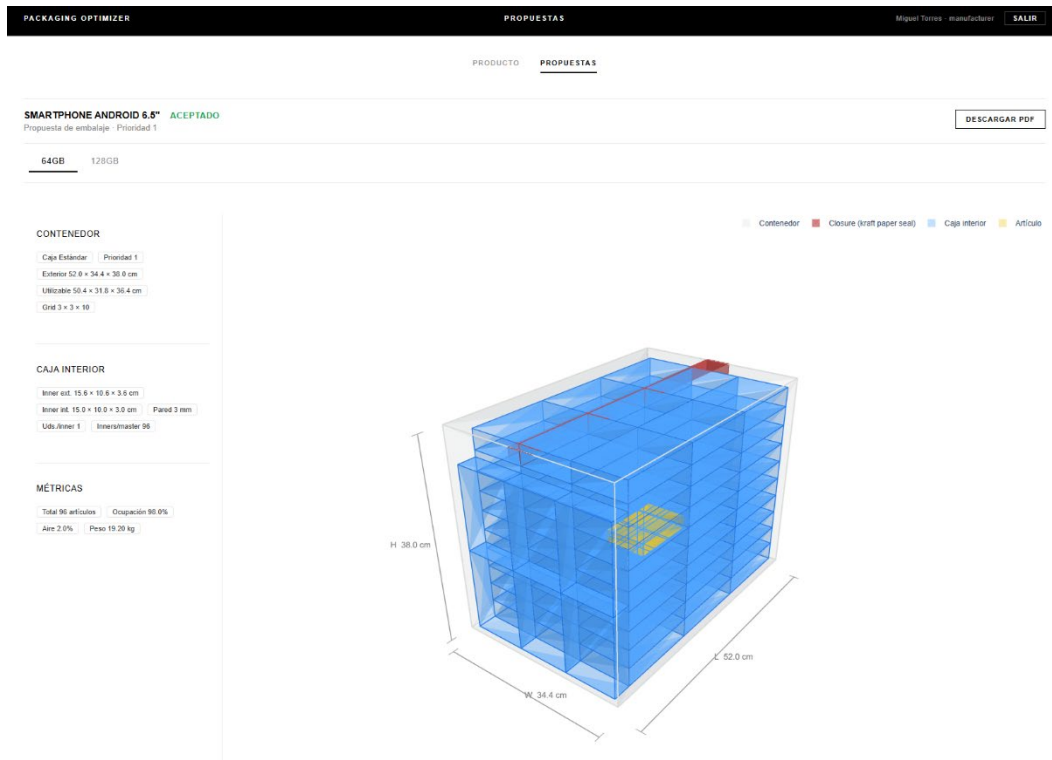


Figura 5: Paso 3. Resultado de optimización.

Precondiciones:

- El usuario está autenticado en el sistema.
- El producto sobre el que se desea realizar la optimización está disponible.
- Se dispone de información de dimensiones y características del producto.

Flujo principal:

1. El usuario accede a la funcionalidad de optimización.
2. El sistema presenta un formulario para introducir los datos del producto.
3. El usuario selecciona el producto y especifica sus características (dimensiones, talla, lote, etc.).
4. El usuario solicita la optimización.
5. El sistema procesa la información y ejecuta el motor de optimización.
6. El sistema determina la mejor configuración de embalaje en base a los contenedores disponibles y las reglas definidas.
7. El sistema genera una representación tridimensional de la solución y calcula las métricas asociadas.
8. El sistema muestra el resultado al usuario.
9. El usuario analiza la propuesta generada.
10. El usuario decide aceptar o rechazar la propuesta.
11. El sistema registra la decisión del usuario.

Flujos alternativos:

- No existe ningún contenedor válido: El sistema informa al usuario de que no se ha encontrado una solución válida.
- El usuario decide rechazar la propuesta: El sistema registra el rechazo y mantiene la trazabilidad de la decisión.

Postcondiciones:

- Se genera una propuesta de embalaje.
- La propuesta queda registrada en el sistema con estado “pendiente”, “aceptada” o “rechazada”.
- Se mantiene la trazabilidad de la decisión tomada por el usuario.

2.4.2. Caso de uso: Revisar propuesta

Actor principal: Gestor de producto / operaciones

Descripción:

Este caso de uso permite al gestor consultar las propuestas de embalaje generadas por los fabricantes, revisar su estado y analizar la información asociada con el fin de realizar un seguimiento operativo del sistema.

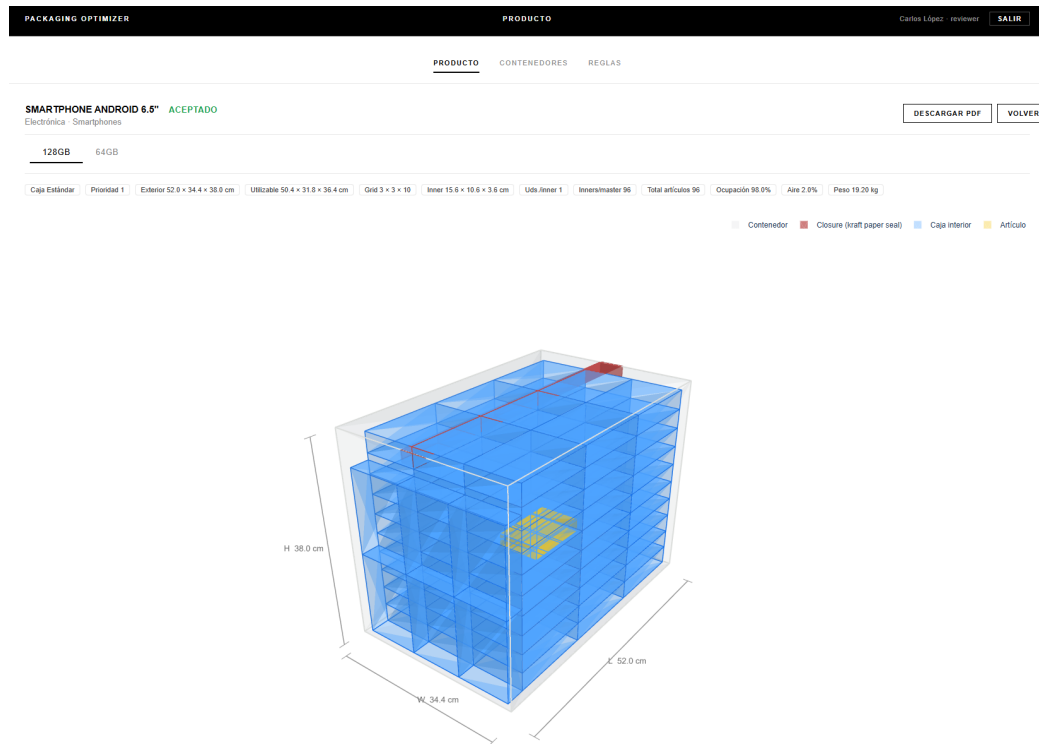


Figura 6: Paso 1: Revisión de propuesta.

Precondiciones:

- El usuario está autenticado en el sistema.
- Existen propuestas de embalaje previamente generadas.

Flujo principal:

1. El usuario accede al listado de propuestas.
2. El sistema muestra las propuestas disponibles junto con su estado.
3. El usuario selecciona una propuesta concreta.
4. El sistema muestra el detalle de la propuesta, incluyendo la representación tridimensional y las métricas asociadas.
5. El usuario consulta la información para realizar el seguimiento o análisis de la propuesta.

Flujos alternativos:

- No existen propuestas disponibles: El sistema informa al usuario de que no hay propuestas para revisar.

Postcondiciones:

- La propuesta permanece registrada en el sistema con su estado correspondiente.
- Se mantiene la trazabilidad de la consulta realizada por el gestor.

2.4.3. Caso de uso: Gestionar contenedores

Actor principal: Gestor de producto / operaciones

Descripción:

Este caso de uso permite al gestor administrar los distintos tipos de contenedores disponibles en el sistema, definiendo sus características físicas y parámetros operativos que serán utilizados posteriormente por el motor de optimización.

Precondiciones:

- El usuario está autenticado en el sistema.
- El usuario dispone de permisos de gestión sobre los contenedores.

Flujo principal:

1. El usuario accede a la funcionalidad de gestión de contenedores.
2. El sistema muestra el listado de contenedores existentes.
3. El usuario selecciona una operación a realizar (alta, modificación, eliminación o consulta).
4. En caso de alta o modificación, el sistema presenta un formulario para introducir o editar los datos del contenedor.
5. El usuario introduce o modifica la información (dimensiones, peso máximo, prioridad, restricciones, etc.).
6. El usuario confirma la operación.
7. El sistema valida los datos introducidos.
8. El sistema guarda la información en la base de datos.
9. El sistema actualiza el listado de contenedores.

Flujos alternativos:

- Datos inválidos: El sistema muestra un mensaje de error y solicita la corrección de los datos.
- Eliminación de contenedor en uso: El sistema impide la operación o muestra una advertencia al usuario.

Postcondiciones:

- El catálogo de contenedores queda actualizado en el sistema.
- Los cambios realizados se reflejan en futuras optimizaciones.

2.5. Diseño del sistema

El diseño del sistema se ha definido a partir de los requerimientos funcionales y del análisis de los casos de uso principales. En esta fase se concreta la estructura interna de la solución, así como la organización de sus entidades, componentes y relaciones, con el objetivo de garantizar la coherencia entre la lógica de negocio, la persistencia de datos y la interacción con el usuario.

Dado que el sistema se concibe sobre una arquitectura de microservicios, el diseño se ha organizado de forma modular, separando las responsabilidades entre los distintos servicios del backend y la capa de presentación. Esta separación permite mejorar el mantenimiento, facilitar la evolución futura del sistema y mantener un desacoplamiento adecuado entre componentes.

En este apartado se describen el modelo de datos, las clases principales del dominio y la arquitectura general de la solución. Estos elementos se complementan con los diagramas correspondientes, que permiten visualizar de forma sintética la estructura del sistema.

2.5.1. Modelo de datos

El sistema utiliza una base de datos orientada a documentos basada en MongoDB, lo que permite representar estructuras flexibles y adaptables a la variabilidad de los datos gestionados.

Dado el enfoque de microservicios adoptado, cada servicio del sistema gestiona su propia base de datos o conjunto de colecciones de forma independiente, evitando acoplamientos innecesarios entre dominios funcionales. Las relaciones entre entidades de distintos servicios no se resuelven mediante joins de base de datos, sino mediante llamadas a través de API REST.

El modelo de datos se organiza en tres bloques principales, correspondientes a los servicios que componen el sistema:

Servicio de usuarios (users-service)

Este servicio gestiona la información de los usuarios del sistema y su control de acceso.

- **User:** Incluye identificador, nombre de usuario, correo electrónico, datos personales, contraseña cifrada y rol asociado (fabricante, gestor o administrador).

Servicio de productos (product-service)

Este servicio gestiona la información estructural de los productos y su clasificación.

- **Family:** Representa la categoría principal del producto.
- **Subfamily:** Permite una clasificación más específica dentro de una familia.
- **Campaign:** Representa la campaña o temporada del producto.
- **Product:** Entidad principal que incluye nombre, descripción, código EAN, referencia al fabricante (User) y relaciones con Family, Subfamily y Campaign.
- **Size:** Representa variantes del producto (por ejemplo, tallas). Se modela como una estructura embebida dentro de Product.

Servicio de optimización (optimization-service)

Este servicio gestiona la información necesaria para el cálculo de las propuestas de embalaje.

- **Container:** Define los contenedores disponibles, incluyendo dimensiones definidas como rangos por eje, peso máximo, prioridad y estado de activación.
- **Rule:** Representa una restricción de embalaje general.

- **RuleAssignment:** Relaciona reglas con familias o subfamilias de productos, permitiendo aplicar restricciones específicas en función del tipo de artículo.
- **LocalRule:** Restricción embebida directamente en un contenedor. Su ámbito se limita a ese contenedor concreto, independientemente de la familia del producto.
- **Proposal:** Almacena las propuestas generadas por el sistema, incluyendo los datos de entrada, los resultados del cálculo, el estado de validación y la referencia al usuario que la generó.

Este modelo permite gestionar de forma eficiente la información necesaria para el proceso de optimización, manteniendo la separación entre servicios y facilitando la evolución independiente de cada componente del sistema.

Las relaciones representadas en el diagrama se implementan mediante referencias por identificador (UUID) en lugar de claves foráneas relacionales, siguiendo el modelo de documentos de MongoDB. Las estructuras marcadas como composición (Size, LocalRule, ArticleDims) se almacenan como subdocumentos embebidos.

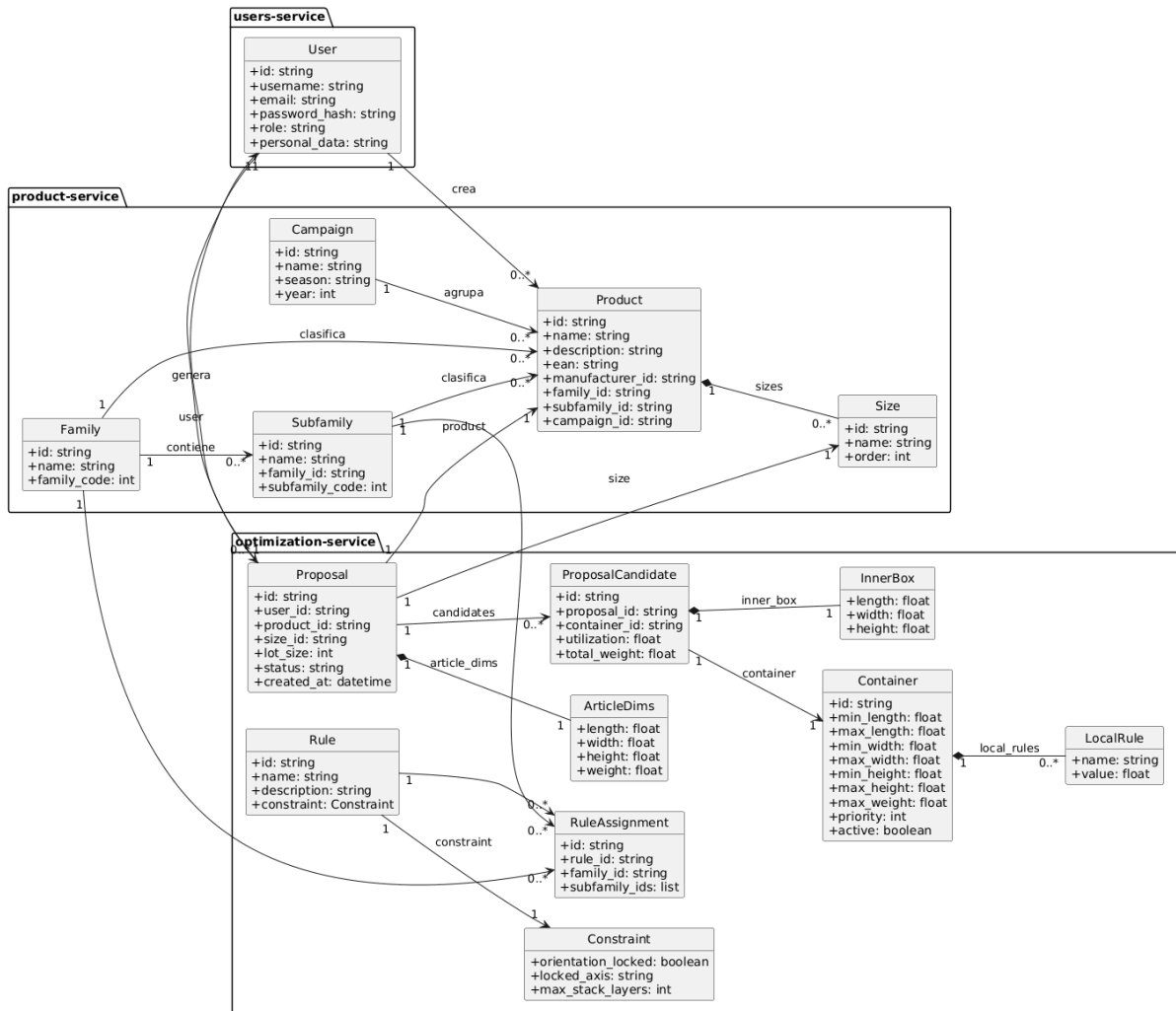


Figura 8: Diagrama de clases de dominio

2.5.2. Clases principales del dominio

Las clases del dominio se definen de forma independiente a la persistencia, centradas en representar la lógica de negocio del sistema y el proceso de optimización tridimensional.

Aunque en la implementación cada microservicio gestiona su propio conjunto de datos, el modelo de dominio se describe de forma unificada para reflejar el comportamiento global del sistema.

El dominio incorpora entidades de usuario, producto y optimización, que se relacionan entre sí para permitir el cálculo, validación y trazabilidad de cada propuesta.

Dominio de producto

- **User:** Representa al usuario que interactúa con el sistema y que puede actuar como fabricante, gestor o administrador.
- **Product:** Representa el artículo a optimizar, incluyendo sus datos identificativos y su clasificación.
- **Size:** Representa una variante del producto, permitiendo modelar distintas configuraciones, por ejemplo tallas.
- **Family / Subfamily:** Permiten clasificar los productos y sirven como base para la aplicación de reglas de embalaje.

Dominio de optimización

- **ArticleDims:** Valor de entrada que recoge las dimensiones físicas del artículo (longitud, ancho, alto y peso). Se almacena como información embebida dentro de Proposal.
- **InnerBox:** Representa la caja interior calculada durante el proceso de optimización.
- **ProposalCandidate:** Modela una posible solución evaluada por el sistema, incluyendo configuración y métricas.
- **Proposal:** Representa la propuesta final generada por el sistema, incluyendo el resultado seleccionado, su estado y la referencia al usuario que la generó.
- **Container:** Representa el contenedor utilizado en la optimización, incluyendo dimensiones, peso máximo y prioridad.
- **Rule:** Define una restricción general de embalaje.
- **RuleAssignment:** Asocia reglas a familias o subfamilias de productos.
- **LocalRule:** Restricción específica asociada directamente a un contenedor.

La relación entre estas clases permite representar el flujo completo de optimización, desde la entrada de datos hasta la generación y validación de una propuesta final.

La separación entre Rule y RuleAssignment permite desacoplar la definición de restricciones de su aplicación concreta, mientras que el uso de ArticleDims como valor embebido en Proposal garantiza la trazabilidad del proceso de cálculo.

A continuación, se muestra el diagrama de clases del dominio que representa las entidades principales del sistema y sus relaciones.

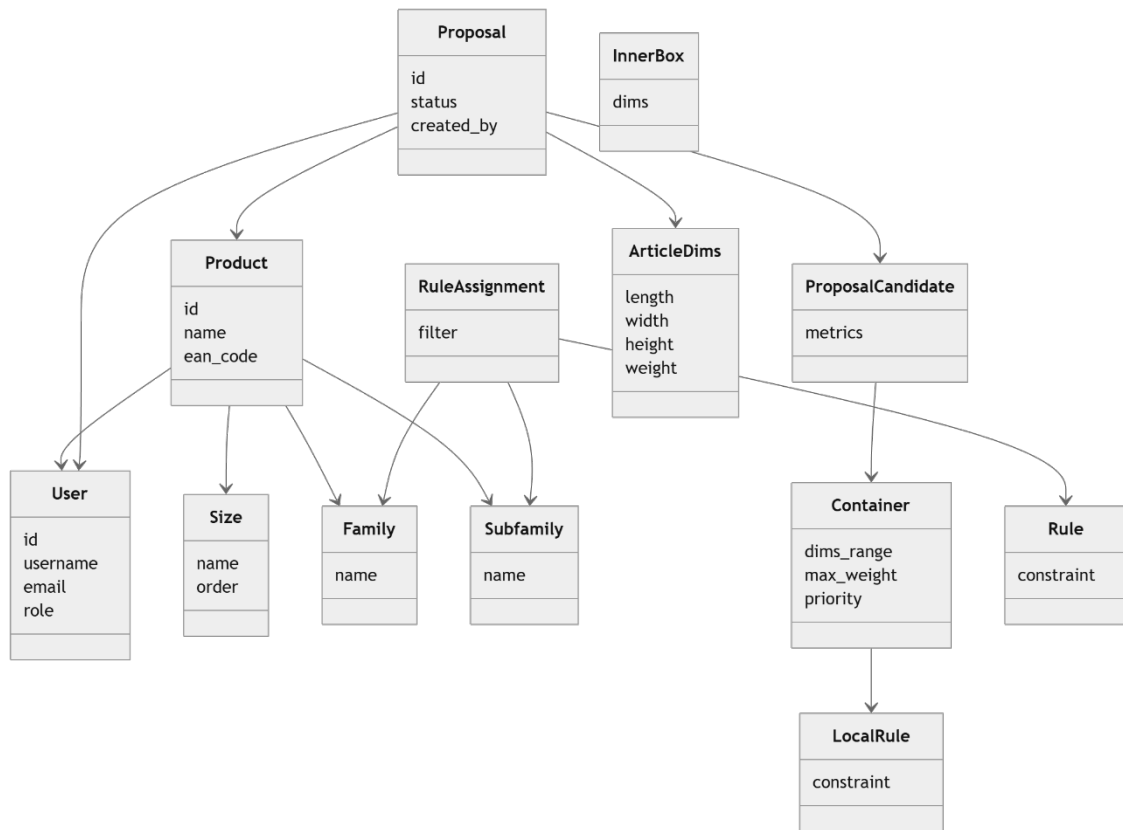


Figura 8: Diagrama de clases de dominio

2.5.3. Arquitectura del sistema

El sistema se ha diseñado siguiendo una arquitectura basada en microservicios, con una separación clara entre la capa de presentación, los servicios de negocio y la persistencia de datos. Este enfoque permite desacoplar los distintos componentes, facilitar su mantenimiento y permitir la evolución independiente de cada uno de ellos.

La solución se estructura en tres capas principales:

Capa de presentación (Frontend): Implementada mediante una aplicación web desarrollada en React. Esta capa actúa como punto de interacción con el usuario y se encarga de gestionar la autenticación, la visualización de datos y la ejecución de operaciones a través de llamadas a los distintos servicios backend.

Capa de servicios (Backend): Compuesta por tres microservicios independientes:

- **users-service:** Encargado de la autenticación y gestión de usuarios, incluyendo la emisión y validación de tokens JWT y el control de acceso basado en roles (RBAC).
- **product-service:** Responsable de la gestión del catálogo de productos, incluyendo familias, subfamilias, campañas y variantes.
- **optimization-service:** Microservicio principal del sistema. Gestiona el proceso de optimización, incluyendo contenedores, reglas, asignaciones de reglas y propuestas generadas. Además, integra el motor de cálculo, la generación de visualización tridimensional y la generación de informes en formato PDF.

Cada microservicio gestiona su propia base de datos MongoDB, manteniendo el desacoplamiento entre dominios funcionales.

Capa de persistencia (MongoDB): Cada servicio dispone de su propia base de datos, evitando dependencias entre servicios y facilitando la escalabilidad del sistema.

El sistema se ejecuta en un entorno containerizado mediante Docker Compose, que actúa como capa de orquestación. Esta herramienta permite desplegar y coordinar todos los servicios del sistema (frontend, microservicios y bases de datos) en un entorno unificado, garantizando la reproducibilidad del sistema y simplificando su configuración.

Adicionalmente, el sistema contempla la integración de un componente de notificación mediante correo electrónico, que se activa cuando una propuesta de embalaje es rechazada. Este mecanismo permite informar automáticamente a los responsables operativos para su revisión. Aunque esta funcionalidad puede estar en desarrollo, forma parte del diseño arquitectónico del sistema.

La comunicación entre el frontend y los microservicios se realiza mediante llamadas REST autenticadas mediante JWT. Cada servicio valida de forma

independiente los tokens recibidos, evitando dependencias directas entre servicios.

Este diseño permite garantizar:

- Desacoplamiento entre componentes
- Escalabilidad horizontal
- Sustituibilidad de servicios
- Separación de responsabilidades

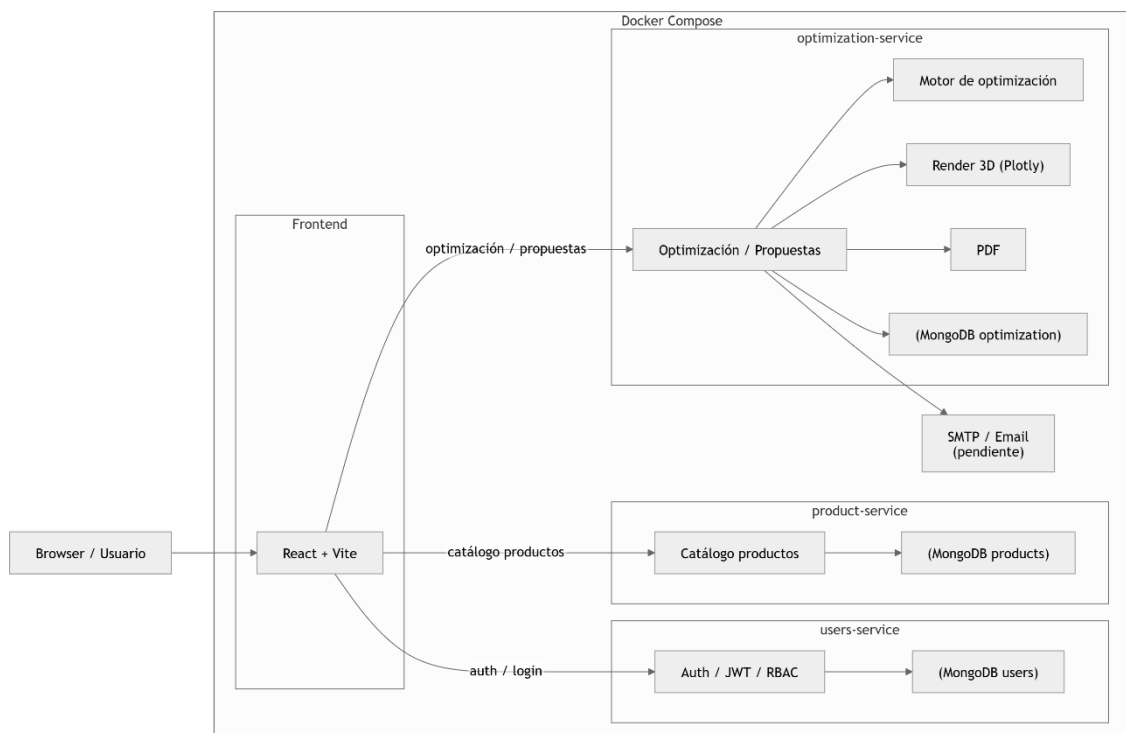


Figura 9: Diagrama de arquitectura